Assignment 4 - Facade Pattern

Home Theater Facade | Software Design Patterns (ShP-2216) | Java 17

Student: [Your full name]

1. Introduction

For this assignment I chose Option B, Home Theater Facade. I wanted to use an example where one simple user action actually requires several smaller operations. Watching a movie sounds like a single task, but the screen has to go down, the lights should be dimmed, and the projector, amplifier and DVD player all need to be prepared in the right order.

Without a facade, the client would have to know which device should be started first and which settings are valid. This is easy to get wrong. For example, the DVD player cannot play without a disc, the amplifier needs an input before its volume is configured, and the projector requires a video input before switching to widescreen mode.

I implemented five subsystem classes: Amplifier, DvdPlayer, Projector, Screen and RoomLights. They store real state and check whether an operation is allowed. HomeTheaterFacade coordinates these classes through two public methods: watchMovie(String title) and endMovie(). Main is the client and only works with the facade. The project is a state-based simulation, not a controller for physical hardware.

2. UML Class Diagram

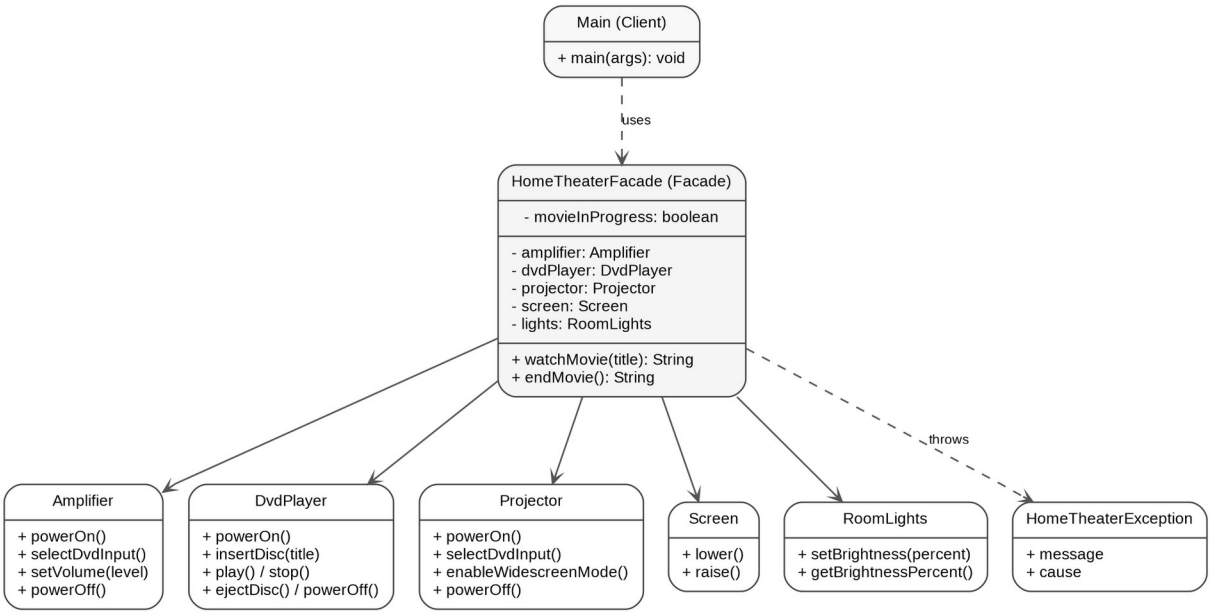


Figure 1. The client uses the facade, which coordinates five independent subsystem classes. The dashed relation also shows the facade exception.

The diagram shows the main dependency direction: Main knows only HomeTheaterFacade. The facade has references to the five subsystems and decides in what order to call them. There are no references from subsystem classes back to the facade. That is important because an advanced client can still use the devices directly when it needs more control.

3. Clean Code Requirements

I focused on Chapters 7, 8 and 9 of Clean Code: Error Handling, Boundaries and Unit Tests. The “before” snippets below are small examples of approaches I avoided, not copies of earlier versions of the project. The “after” snippets are taken from the implementation, with some surrounding lines omitted.

3.1. Fail fast with a useful exception (Chapter 7)

Returning false for invalid input would not tell the client what went wrong. I validate the title before touching any device. An IllegalArgumentException gives a clear reason, and the room stays unchanged.

Before (example of what I avoided):

```
if (title.isBlank()) return false;
```

After (from the implementation):

```
if (title == null || title.isBlank()) {
    throw new IllegalArgumentException("Movie title must not be blank.");
}
```

3.2. Preserve error context and recover safely (Chapter 7)

If a device fails during setup, the facade tries to return the theater to standby. It then throws a HomeTheaterException that says which movie could not start and keeps the original exception as its cause. If cleanup also fails, that extra failure is attached as suppressed instead of being silently discarded.

Before (example of what I avoided):

```
catch (RuntimeException problem) {
    throw new RuntimeException("Failed");
}
```

After (from the implementation):

```
catch (RuntimeException problem) {
    restoreStandby(problem);
    throw new HomeTheaterException(
        "Could not start movie '" + movieTitle + "'.", problem);
}
```

3.3. Keep device coordination at one boundary (Chapter 8)

Chapter 8 is about working with boundaries cleanly. In this project the boundary is between the client and the device subsystem. The client should not repeat the setup sequence or depend on individual device APIs; those calls belong in the facade.

Before (example of what I avoided):

```
screen.lower();
lights.setBrightness(25);
projector.powerOn();
// ... many more calls in Main
```

After (from the implementation):

```
HomeTheaterFacade theater = new HomeTheaterFacade();
theater.watchMovie("Interstellar");
```

3.4. Do not expose subsystem objects in the public API (Chapter 8)

If watchMovie returned DvdPlayer, the client would become coupled to one internal class. I made the public result a simple confirmation message. A caller does not need to understand how the DVD player works to use the feature.

Before (example of what I avoided):

```
public DvdPlayer watchMovie(String title);
```

After (from the implementation):

```
public String watchMovie(String title);
// Example result: "Now playing: Interstellar"
```

3.5. Give each test one clear purpose (Chapter 9)

I separated tests by behavior instead of creating one long test that checks everything. This makes a failing test easier to understand. For example, one test verifies that an invalid movie title does not change any device state.

Before (example of what I avoided):

```
void testEverything() {
    // startup, validation, shutdown and failures
}
```

After (from the implementation):

```
private static void blankTitleDoesNotStartTheater() {
    Fixture f = new Fixture();
    expect(IllegalArgumentException.class,
           () -> f.facade.watchMovie("  "));
    assertStandby(f);
}
```

3.6. Make tests fast, independent and self-checking (Chapter 9)

The tests follow the useful parts of the FIRST rules. They do not require physical equipment, files, network access or third-party libraries. Each test receives fresh components and uses checks that fail automatically if behavior is wrong. This prevents one test from leaving state that changes another test.

Before (example of what I avoided):

```
static HomeTheaterFacade shared = new HomeTheaterFacade();
// Different tests modify the same object.
```

After (from the implementation):

```
Fixture f = new Fixture();
f.facade.watchMovie("Interstellar");
check(f.dvdPlayer.isPlaying(), "DVD is not playing");
```

4. Conclusion

The main benefit of my Home Theater Facade is that it hides the sequence of device setup and shutdown. Main only asks the system to start or end a movie. It does not need to know about power states, input selection, brightness, volume, disc insertion or error recovery.

At the same time, the facade does not remove access to the individual devices. Amplifier, DvdPlayer, Projector, Screen and RoomLights are still public, independent classes. I also kept the facade responsible for coordination rather than moving all device behavior into it.

I compiled the code with the Java 17 language target and ran nine automated tests. All nine passed, including a simulated projector failure during setup. In a real application, device failures would need additional hardware-specific handling, but for this assignment the simulation shows why a single coordination point is useful.

5. GitHub Repository

Repository link: [Paste your GitHub repository URL here after publishing]

The project includes source code, nine tests, a README with run instructions, this UML diagram and incremental Git commits. The GitHub address needs to be added after the repository is pushed.